

Reviewer Pack — Kronecker Coefficient Growth Separates Permanent vs Determinant

Entry 488f1af8a053 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 20:40:48 UTC

Kronecker Coefficient Growth Separates Permanent vs Determinant

Entry ID: 488f1af8a053 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 20:40:48 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Permanent vs Determinant Complexity

Statement:

For all $n \geq 2$, the maximum Kronecker coefficient in the decomposition of $\text{Sym}^n(\text{Sym}^k(V))$ grows polynomially in n for determinants but exponentially in n for permanents, where V is the standard n -dimensional representation of $\text{GL}_n(\mathbb{C})$.

Rationale (proposer's reasoning):

Schur-Weyl duality decomposes tensor products into irreducible representations, and Kronecker coefficients count multiplicities. The permanent's asymmetric decomposition (via symmetric powers) suggests exponential complexity, while determinants' symmetric structure implies polynomial bounds. This bridges representation theory to algebraic complexity.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6378756f5a653317

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def hook_length_formula(shape):
    n = len(shape)
    result = 1
    for i in range(n):
        for j in range(len(shape[i])):
            result *= (shape[i][j] + i - j) / (i + j + 1)
    return result

def kronecker_coefficient(lam, mu):
    n = len(lam)
    m = len(mu)
    if sum(lam) != sum(mu):
        return 0
    numerator = 1
    for i in range(n):
        for j in range(m):
            numerator *= hook_length_formula(lam[:i+1]) * hook_length_formula(mu[:j+1])
    denominator = math.factorial(sum(lam)) * math.prod(math.factorial(lam[i] + mu[j] - lam[i][j]))
    return numerator // denominator

def run_trial(seed: int):
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            k = random.randint(1, n-1)
            shape = [(n-k), (k)]
            kronecker_coeffs = [kronecker_coefficient(shape[:i+1], shape[i+1:]) for i in range(len(shape))]
            max_kronecker_coeff = max(kronecker_coeffs)
            total_metric_value += max_kronecker_coeff
```

```

        instances_tested += 1

mean_metric_value = total_metric_value / instances_tested
if n_values[-1] == 40 and mean_metric_value > 1e6:
    conjecture_holds = False
    counterexample = "Exponential growth observed for permanents."

return {
    "metric_name": "Max Kronecker Coefficient",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3**j for i in range(5) for j in range(5)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.6f} std=0.000000 support_fraction=1.000000")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.6f} std=0.000000 support_fraction={support_fraction:.6f}")
    else:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"Exponential growth observed for permanents.\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 11, "metric_name": "Max Kronecker Coefficient", "metric_value": 0.000000, "instances_tested": 25, "conjecture_holds": false, "counterexample": "Exponential growth observed for permanents."}
TRIAL: {"seed": 23, "metric_name": "Max Kronecker Coefficient", "metric_value": 0.000000, "instances_tested": 25, "conjecture_holds": false, "counterexample": "Exponential growth observed for permanents."}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f418874.py", line 74, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f418874.py", line 50, in run_trial
    kronecker_coeffs = [kronecker_coefficient(shape[:i+1], shape[i+1:]) for i in range(len(shape))]
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f418874.py", line 34, in kronecker_coefficient
    numerator *= hook_length_formula(lam[:i+1]) * hook_length_formula(mu[:j+1])
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f418874.py", line 22, in hook_length_f
    for j in range(len(shape[i])):
                   ^^^^^^^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to a `TypeError` in `hook_length_formula`, preventing reliable evaluation of the conjecture | next: Fix the shape handling in `hook_length_formula` to properly process integer inputs as list-like objects

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	70793
2	preregistration	ollama_remote	qwen3:8b	0	0	24337
3	novelty	ollama_remote	qwen3:8b	0	0	20750
4	novelty	ollama_remote	qwen3:8b	0	0	12657
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12765
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9829
7	judge	ollama_remote	qwen3:8b	0	0	18671

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 169803 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/488f1af8a053.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/488f1af8a053.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/488f1af8a053.tar.gz` (if generated)